

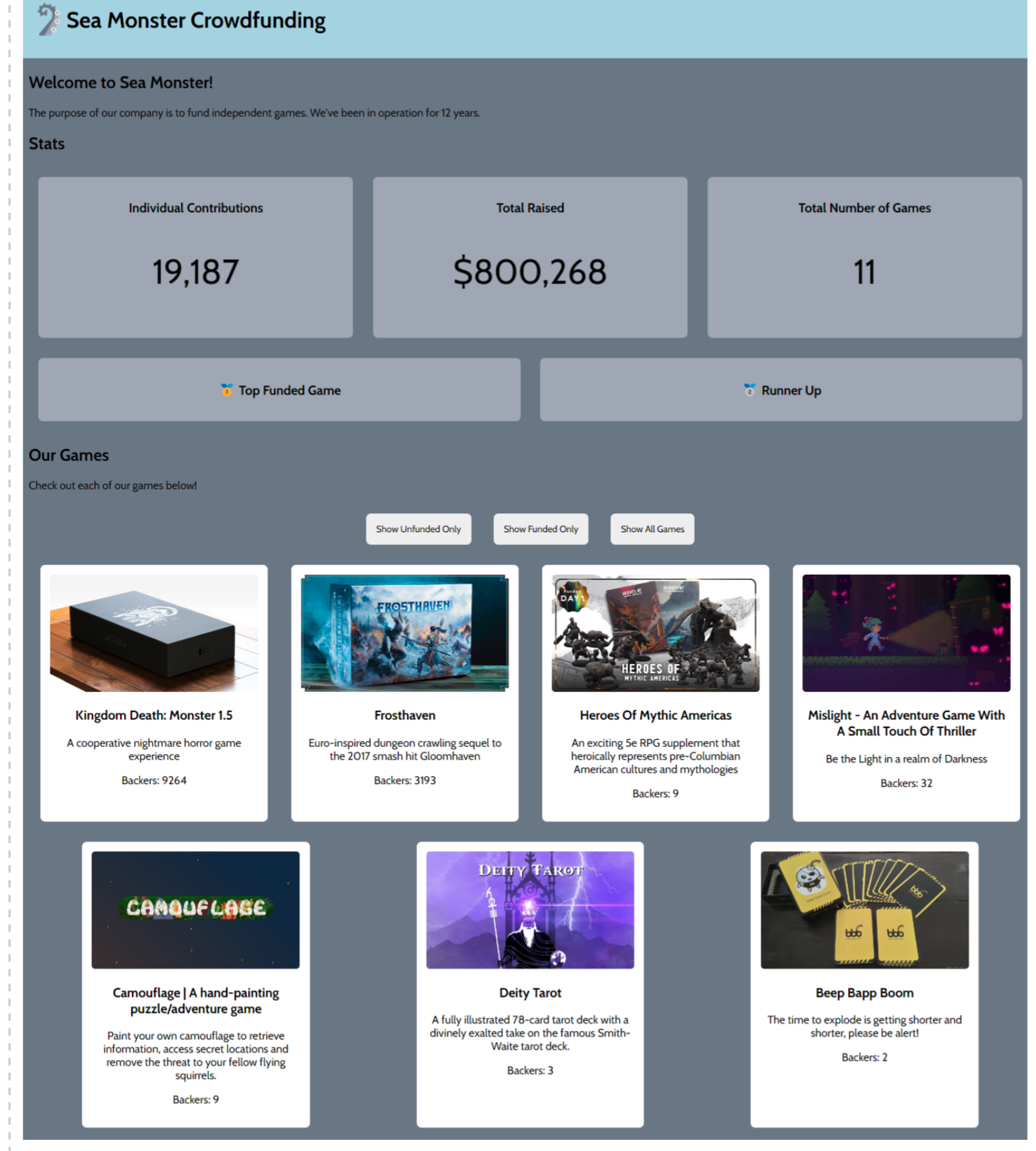
WEB102 Prework

Challenge 6 of 7: Add more information at the top of the page about the company

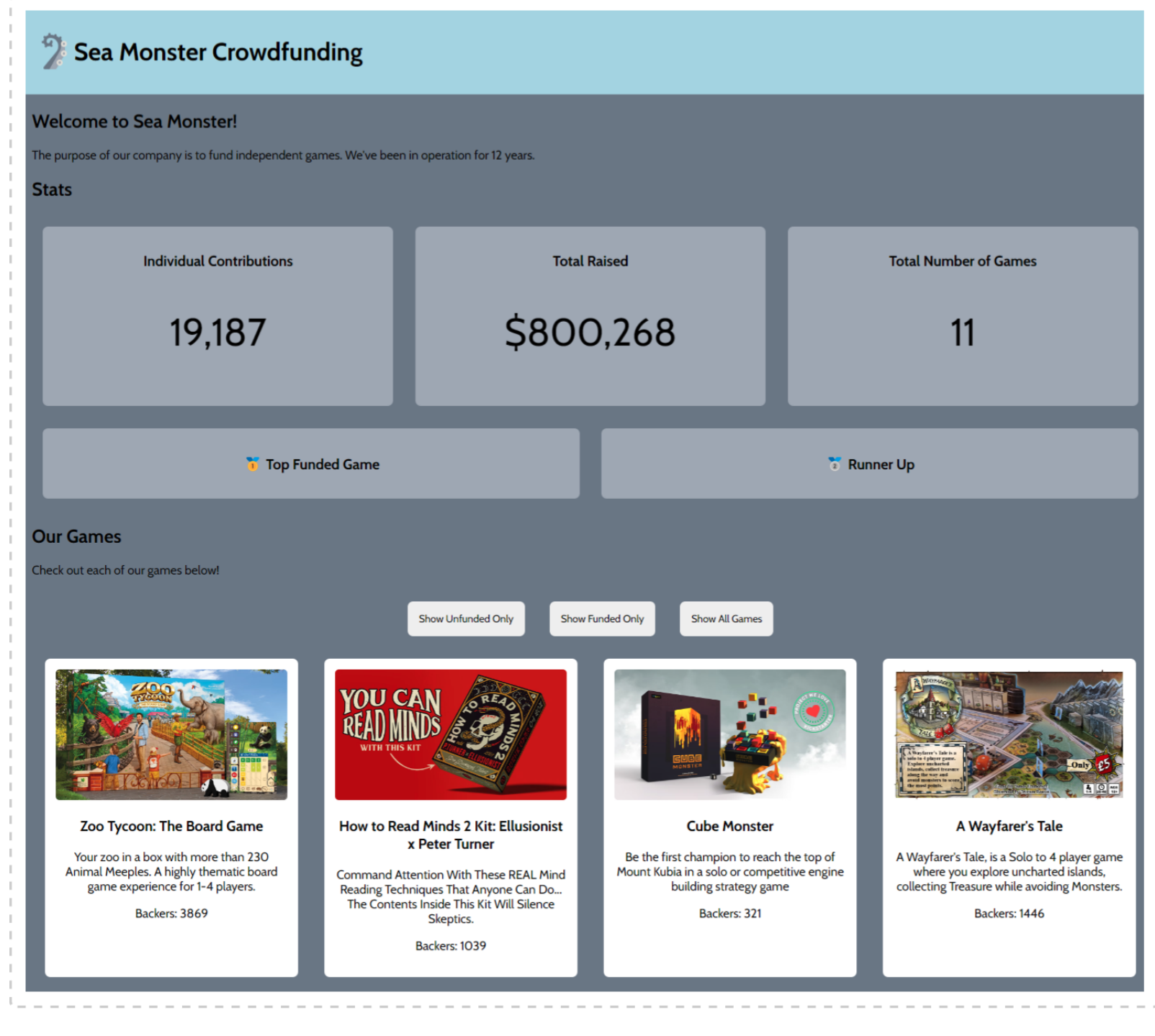
✔ Checkpoint

Wow, challenge 6 already? You're crushing these challenges!

At this point, your website should look something like the screenshot below when the "Show Unfunded Only" filter is selected:



And it should look roughly like the screenshot below when the "Show Funded Only" filter is selected:



Compare your work with the examples above and see if there are any changes you might need to make!

📺 Background

JavaScript often offers multiple ways to accomplish the same task, such as the two different types of functions we discussed earlier. Another example of flexible implementation in JavaScript is the ternary operator, which looks like a question mark (?). (Ternary comes from the Latin word for three.) The ternary operator functions as a compact if-else statement. It has three parts: the condition to check, the result to return if the condition is true, and the result to return if the condition is false.

Let's use another Spotify example. Imagine we want to show the artist of a particular song if the artist is known, and if not, we want to display unknown. Here's how we might do that with an if statement:

```
let artistStr = "The artist is: ";

if (song.hasArtist()) {
  artistStr += song.artist;
} else {
  artistStr += "Unknown artist";
}
```

Here's what this same process would look like with the ternary operator:

```
let artistStr = `The artist is: ${song.hasArtist() ? song.artist : "Unknown artist"}`;
```

You can see the ternary operator is short and sweet!

If the code segment above seems pretty abstract, try reading it in your head like this: "Does the song have an artist? If so, put the song's artist here. If not, just say unknown artist instead."

Ternary operators are highly useful within template strings or in other places where a large if-else block is inconvenient. They are often seen in React, the frontend framework we will use throughout this course, for example in a process called "conditional rendering," where parts of the user interface are loaded only if certain conditions are met.

🎯 Challenge Steps

➡️➡️➡️ Step 1

Use **filter** or **reduce** to sum the number of unfunded games in **GAMES_JSON**.

➡️➡️➡️ Step 2

Use a template string to display how much money has been raised and for how many games, as well as explaining how many games currently remain unfunded. Use the ternary operator (?) to make sure the statement is grammatically correct regardless of the number of unfunded games. Here's an example format:

```
11 // one game is unfunded
12 const displayStr =
13   'A total of $100,000 has been raised for 4 games. Currently, 1 game
14   remains unfunded. We need your help to fund these amazing games!'
15 // any other number of games is unfunded
16 const displayStr =
17   'A total of $100,000 has been raised for 4 games. Currently, 2 games
18   remain unfunded. We need your help to fund these amazing games!'
```

Hint: consider what sections of the string above need to change, and what condition should change that section.

🔑 **Secret Key component 1:** Which function adds correct commas and punctuation to a number, e.g. displaying **100000** as **100,000**? (Use only the name of the function with **no punctuation**. For example, if the answer was **filter**, use **filter**, not **.filter** or **.filter()**).

➡️➡️➡️ Step 3

Create a new paragraph element containing the template string you created and add the element to the **descriptionContainer**.

🔑 **Secret Key component 2:** What is the opening tag of the HTML element you appended your newly created **<p>** element to? (Use the full opening tag with angle brackets. For example, if the answer was a **<p>** element, use **<p>**.)

🔑 **Secret Key component 3:** Which option would correctly display "Hello {name}" if a user is logged in, and simply "Hello!" if the user is not logged in?

1. `isLoggedIn ? `Hello ${user.name}` : "Hello!"`; (keyword: 1)
2. `isLoggedIn : "Hello ${user.name}" : "Hello!"`; (keyword: 2)
3. `isLoggedIn ? "Hello!" : `Hello ${user.name}``; (keyword: 3)
4. `if isLoggedIn ? `Hello user.name` else "Hello!"` (keyword: 4)

🔑 **Secret Key component 4:** What does the following code segment return when called with **true**?

```
function getFee(isMember) {
  return (isMember ? '$2.00' : '$10.00');
}
```

1. **\$10.00** (keyword: GAP)
2. **\$2.00** (keyword: IVY)
3. **2.0** (keyword: VAN)
4. **Null** (keyword: ICE)

🚀 Moving on to Challenge 7

Create the 🔑 **Secret Key** from its components with no spaces in between:

- Component 1: Which function adds correct commas and punctuation to a number, e.g. displaying **100000** as **100,000**?
- Component 2: What is the opening tag of the HTML element you appended your newly created **<p>** element to?
- Component 3: Which option would correctly display "Hello {name}" if a user is logged in, and simply "Hello!" if the user is not logged in?
- Component 4: What does the following code segment return when called with **true**?

Hint: your 🔑 **Secret Key** should be 23 characters long.

Reminders about 🔑 **Secret Keys**:

- Keys for the Offline PDF version of prework **are case sensitive!**
 - (For example, "SEA_MONSTER" is NOT the same as "sea_monster")
- Secret keys can be any combination of letters, numbers, and symbols.

Use your 🔑 **Secret Key** to unlock the next PDF file. If your 🔑 **Secret Key** is correct, you will move onto the next step!